In production, you want the web server to be responsible for hosting static files; it is just far more efficient. The settings in the example are the same, but you also need to configure your web server. Inside the web server, you map URLs to directories, so /static/ maps to where your static files are located.

Web server permissions are typically associated with the directory of the files being served. You don't want your Django code served by the web server, only the static files. For this reason (and another I'll get to in a second), it is best practice to copy the static files somewhere outside the project directory for serving by the web server.

Django provides a management command for copying the static files to their production location. This command is `collectstatic`. As part of your deployment process, you run this command, and then Django copies all the static files to the production location. You inform Django of the destination location by setting `STATIC_ROOT`, as in the following example:

```
# RiffMates/RiffMates/settings.py
...
STATIC_ROOT = OUTSIDE_DIR / 'static'
```

In chapter 7, you learned how I typically have a directory named *outside* at the same level as the project directory. I put user-uploaded files in this directory, and in the example configuration, I've done the same with the production static files.

How static files get managed is determined by a pluggable component. Web servers—and web browsers, for that matter—like to cache things. If you update your site's logo, just because you replaced it on the web server doesn't mean it will get served or the browser will even ask for it; it may use a cached copy. Enter `ManifestStaticFilesStorage`. The class name may be a mouthful, but what it does is automatically rename your static files to include their MD5 hash. An MD5 hash is a short string that represents the content of a file: changing the file changes the hashed string. By appending the hash to the filename, any change to the file changes the filename, meaning it won't be cached. As you're using the `{% static %}` tag to serve your static files, Django takes care of name translation in the URL for you.

Django allows for the customization of other kinds of storage handlers as well, and when configuring to use the MD5 static handler, you have to be careful not to overwrite the other settings. To use the MD5 static handler, include all the following in settings.py:

```
# RiffMates/RiffMates/settings.py
...
STORAGES = {
  "default": {
      "BACKEND": "django.core.files.storage.FileSystemStorage",
  },
  "staticfiles": {
    "BACKEND": \
        "django.contrib.staticfiles.storage.ManifestStaticFilesStorage",
  },
}
```